

Introduction

Regular Expression

Python RegEx, or regular expressions, is a sequence of characters used to define a search pattern. It helps us match, search, split, or validate strings based on specific patterns.

The re module in Python supports regular expressions. It allows us to search for strings using patterns and returns the first match or None if no match is found.

Regular expressions in Python are useful for:

- Identify patterns in a string/ file.
- Search for a substring in a string.
- Replace a section of a string with another string.
- Validate email format.
- Split a string into substrings.

Example

```
import re

text = "The rain in Spain"

pattern = "rain"

# Search for the word "rain" in the text

match = re.search(pattern, text)

if match:

    print("Match found!")

else:

    print("No match found.")
```

Output

Match found!

Regex Module in Python

[Python](#) provides a built-in module called re to work with regular expressions. It helps us search, match, and manipulate text using patterns.

The Python regular expression module is a powerful tool for handling text. Once we understand its functions and syntax, we can efficiently perform pattern matching and text processing tasks.

Import the Module

Start by importing the 're' module to use the Python RegEx function

```
import re
```

Basic Functions

- **search():** It is used to search a string for a match. Once found, it returns the matched object.
- **match():** It checks if the start of a string matches the pattern.
- **findall():** It finds all matches in a string and returns a list of them.
- **sub():** It replaces the matches of the pattern with the specified string.

Special Characters

- **\ (Backslash)** escapes metacharacters or signals a particular sequence.
- **^ (Caret)** is used to match the beginning of a string.
- **.** (Dot) matches all characters except a newline.
- **[] (Square Brackets)** matches one of the characters within brackets.
- **\$ (Dollar Sign)** matches the end of the string.

Special Sequences

- **\s** to match a whitespace character.
- **\w** to match an alphanumeric character.
- **\W** to match a non-alphanumeric character.
- **\d** to match a digit.
- **\D** to match a non-digit character.
- **\S** to match a non-whitespace character.

Quantifiers

- **?** to match 0 or 1 repetition of the preceding pattern.
- **{n,}** to match n or more repetitions of the preceding pattern.
- **{n}** to match n repetitions of the preceding pattern.
- **{n,m}** to match between n and m repetitions of the preceding pattern.
- ***** to match 0 or more occurrences of the preceding pattern.
- **+** to match 1 or more occurrences of the preceding pattern.

Compiling Patterns

MetaCharacters	Description
\	Escapes special characters or signals a special sequence
[]	Matches any one character inside the brackets
^	Matches the beginning of a string
\$	Matches the end of a string
.	Matches any character except a newline(\n)
	Matches with any of the characters separated by it.

?	Matches 0 or 1 occurrence of the preceding element
*	Matches 0 or more occurrences
+	Matches 1 or more occurrences
{ }	Specifies the exact numbers or a range of repetitions
()	Groups multiple patterns together

We compile the regular expression (RegEx) pattern using `re.compile()` to enhance performance for patterns used multiple times.

Example:

```
import re

pattern = re.compile(r"\d+")

print(pattern.findall("Price: 100, Discount: 20"))
```

Output:

```
['100', '20']
```

Metacharacters in Python RegEx

Metacharacters are special characters with specific meanings in pattern matching. They help us group, repeat, and define search patterns in text.

Metacharacters are special characters with specific meanings in pattern matching. They help us group, repeat, and define search patterns in text.

Here is the detailed discussion of each metacharacter in RegEx

1. \ – Backslash

Backslash (\), known as the escape character, tells Python to treat a special character (like \$, ., or *) as a normal character. It's used to escape metacharacters or define special sequences like `\d` or `\s`.

Example:

```
import re

text = "Price is $100"

match = re.search(r"\$", text) # Match the $ character using escape

if match:

    print("Matched:", match.group())
```

Output:

```
Matched: $
```

2. [] – Square Brackets

Regular expressions in Python use square brackets [] to define a character set or range.

- Use - to specify a range of characters.
- Use ^ to invert (negate) the set.

Thus:

- [0,3] is the same as [0123]
- [b-d] is the same as [bcd]
- [^0-3] matches anything except 0, 1, 2, or 3
- [^b-d] matches anything except b, c, or d

Example:

```
import re

pattern = r'[a-zA-Z]' # Match any lowercase or uppercase English letter

string = "123 abc XYZ"

matches = re.findall(pattern, string) # Find all matching characters

print("Matches found:", matches)
```

Output:

Matches found: ['a', 'b', 'c', 'X', 'Y', 'Z']

3. ^ – Caret

The caret symbol (^) matches the beginning of a string. This means we can check if a string begins with the mentioned character.

- ^h matches strings like "home", "house", "host"
- ^ho matches strings starting with "ho", like "home" or "host"

Example:

```
import re

pattern = r'^Hello' # Match if the string starts with 'Hello'

string = "Hello, world!"

match = re.match(pattern, string)

if match:

    print("Match found:", match.group())

else:

    print("No match found.")
```

Output:

Match found: Hello

4. \$ – Dollar

The dollar symbol (\$) is used in regular expressions to match the end of a string. It ensures that the string ends with the specified character or pattern.

Example:

```
import re

pattern = r'world!$' # Match 'world!' at the end of the string

string = "Hello, world!"

match = re.search(pattern, string)

if match:

    print("Match found:", match.group())

else:

    print("No match found.")
```

Output:

Match found: world!

5. . – Dot

The dot (.) matches any single character except a newline. It acts as a wildcard in patterns.

- c.d matches "cad", "cbd", "c9d", etc.
- .. checks if the string contains at least two characters

Example:

```
import re

pattern = r'a.c' # Match 'a', any character, then 'c'

string = "abc a1c a2c"

matches = re.findall(pattern, string)

print("Matches found:", matches)
```

Output:

Matches found: ['abc', 'a1c', 'a2c']

6. | – Or

The pipe symbol (|) is used to match either the pattern before or after it. It helps us search for multiple alternatives within a string.

- c|d matches either 'c' or 'd'
- (cat|dog) matches either 'cat' or 'dog'

Example:

```
import re # Importing the regex module

pattern = r'cat|dog' # Matches either 'cat' or 'dog'

string = "I have a cat and a dog."

matches = re.findall(pattern, string)

print("Matches found:", matches)
```

Output:

Matches found: ['cat', 'dog']

7. ? – Question Mark

The question mark (?) symbol is a quantifier in regular expressions in Python, showing that the preceding element must be matched zero or one time. We can specify the element as optional, which means it can occur once or not at all.

- `cd?a` matches "ca" and "cda" (the 'd' is optional)

Example:

```
import re

pattern = r'cd?a' # Matches 'c' followed by 0 or 1 'd', then 'a'

string = "ca cda cdba"

matches = re.findall(pattern, string)

print("Matches found:", matches)
```

Output:

Matches found: ['ca', 'cda']

8. * – Star

The asterisk (*) is a quantifier that matches zero or more repetitions of the character or group before it.

- `cd*a` matches "ca", "cda", "cdda", "cddda", etc.

Example:

```
import re

pattern = r'cd*a' # 'c' followed by 0 or more 'd', then 'a'

string = "ca cda cddda cdba"

matches = re.findall(pattern, string)

print("Matches found:", matches)
```

Output:

Matches found: ['ca', 'cda', 'cddda']

9. + – Plus

The plus (+) is a quantifier in regular expressions. It matches one or more repetitions of the character or group before it.

- `cd+a` matches `"cda"`, `"cdda"`, `"cddda"`, etc.
- It does not match `"ca"` (because at least one 'd' is required)

Example:

```
import re

pattern = r'cd+a' # 'c' followed by 1 or more 'd', then 'a'

string = "ca cda cddda cdba"

matches = re.findall(pattern, string)

print("Matches found:", matches)
```

Output:

Matches found: ['cda', 'cddda']

10. {m, n} – Braces

Braces {m,n} match at least m and at most n repetitions of the preceding character or group.

- For example, `c{1,3}a` matches patterns like `"ca"`, `"cca"`, or `"ccca"`, but not `"a"` or `"cccca"`.

Example:

```
import re

pattern = r'c{1,3}a' # Match 'c' repeated 1 to 3 times followed by 'a'

string = "a ca cca ccca cccca ab"

matches = re.findall(pattern, string) # Find all matches in the string

print("Matches found:", matches)
```

Output:

Matches found: ['ca', 'cca', 'ccca']

11. (<regex>) – Group

We use parentheses () in regular expressions to group subpatterns.

Example:

```
import re

pattern = r'(ab)+' # Group 'ab' and match it one or more times

string = "abab abc ababab"

matches = re.findall(pattern, string) # Find all matching groups
```

```
print("Matches found:", matches)
```

Output:

Matches found: ['ab', 'ab', 'ab']

Special Sequences in Python RegEx

Rather than matching the actual character in the string, special sequences locate the specific position in the string that matches, simplifying the way to write commonly used patterns.

Sequence	Description
\A	Matches if the string starts with the characters to the right of it.
\b	Matches if the pattern is at the beginning or end of a word.
\B	Opposite of \b. Matches only when not at the start or end of a word.
\d	Matches any digit (equivalent to [0-9]).
\D	Matches any non-digit (equivalent to [^0-9]).
\s	Matches any whitespace (space, tab, newline, etc.).
\S	Matches any non-whitespace character.
\w	Matches any alphanumeric character (letters, digits, underscore).
\W	Matches any non-alphanumeric character.
\Z	Matches if the string ends with the characters to the left of it.

Python RegEx Functions

The Python 're' module offers a set of functions used to work with regular expressions and search a string for a matching pattern. Here are a few main functions we use:

Function	Description
re.findall(pattern,string)	The findall() function matches all patterns in the string and returns a list of matches.
re.search(pattern,string)	We use this to search the first match of a pattern in the string. It returns a match object or None if not found.

<code>re.compile()</code>	This function compiles regular expressions into pattern objects for repeated use. It is useful for improving performance when the pattern is repeated.
<code>re.escape(string)</code>	This function escapes all special characters in a string and treats them as literals.
<code>re.split(pattern,string)</code>	This function splits the string and returns a list showing where the string has been split at each match.
<code>re.sub(pattern,rep_substring,string)</code>	It replaces one or more matching patterns in the string with the substring.

Here is a look at the definitions and examples of Python RegEx functions:.

1. `re.findall()`

The `findall()` function in Python searches a string for all parts that match a pattern and returns them as a list. Strings are scanned left to right.

```
import re

# Pattern to find all words in the string

pattern = r'\w+'

text = "Python 3.12 is awesome!"

# Find all words or alphanumeric sequences

matches = re.findall(pattern, text)

# Display the matches

print("Matches found:", matches)
```

Output:

Matches found: ['Python', '3', '12', 'is', 'awesome']

2. `re.search()`

The `search()` function looks through a string and finds the first match of the pattern. It stops at the first match and returns details about it. If nothing matches, it returns `None`.

Example:

```
import re

# Pattern to search for the word 'Python'
```

```

pattern = r'Python'

text = "I love Python and Python is powerful."

# Search for the first match

match = re.search(pattern, text)

# Check and print the result

if match:

    print("First match found:", match.group()) # Output: Python

else:

    print("No match found.")

```

Output:

First match found: Python

3. re.compile()

The compile() function converts a Python regular expression pattern into a reusable object. This object can then be used multiple times to match or search strings, saving time and improving performance.

Example:

```

import re

# Compile the pattern once for reuse

pattern = re.compile(r'\d+') # Matches one or more digits

# Use the compiled pattern to search in strings

match1 = pattern.search("Order number: 12345")

match2 = pattern.search("Invoice: 9876")

# Print matches

print("First match:", match1.group())

print("Second match:", match2.group())

```

Output:

First match: 12345

Second match: 9876

4. re.escape()

The `escape()` function adds a backslash (\) before every special character in a string. We use it when we want to treat all characters as plain text, even if they are RegEx symbols.

Syntax:

```
re.escape(string)
```

Example:

```
import re

text = "Price is $40.99 (tax included)."

# Escape all special characters in the string

escaped_text = re.escape(text)

print("Escaped string:", escaped_text)
```

Output:

Escaped string: Price\ is\ \\$40\.99\ \ (tax\ included\)\.

5. re.split()

The Python RegEx `split` function splits the string by the pattern occurrences. Once the pattern is found, it returns the remaining characters from the string as a list after splitting.

Syntax:

```
re.split(pattern, string, maxsplit=0, flags=0)
```

- **Pattern:** The regex pattern to split on.
- **string:** The string to be split.
- **maxsplit:** (Optional) Maximum number of splits. Defaults to 0 (no limit).
- **flags:** (Optional) Regex flags like `re.IGNORECASE`.

Example:

```
import re

text = "apple, banana; mango orange"

# Split on comma, semicolon, or space

result = re.split(r'[;, ]+', text)

print("Split result:", result)
```

Output:

Split result: ['apple', 'banana', 'mango', 'orange']

6. re.sub()

The `re.sub()` function searches for a regex pattern in a string and replaces it with a specified substring. It's commonly used to clean or modify text.

Syntax:

```
re.sub(pattern, repl, string, count=0, flags=0)
```

- **pattern:** The regex pattern to search.
- **repl:** The replacement string.
- **string:** The input string.
- **count:** (Optional) How many matches to replace. The default is 0 (replace all).
- **flags:** (Optional) Regex flags like `re.IGNORECASE`.

Example:

```
import re

text = "blue sky and blue ocean"

# Replace "blue" with "green"

new_text = re.sub(r'blue', 'green', text)

print("Updated text:", new_text)
```

Output:

Updated text: green sky and green ocean

7. `re.subn()`

The `subn()` function and `sub()` functions are the same. However, they differ in the way they provide output. Unlike `sub()`, which returns only a string, the `subn()` function returns a tuple with total replacement counts and a new string.

Syntax:

```
re.subn(pattern, repl, string, count=0, flags=0)
```

- **pattern:** The regex pattern to match.
- **repl:** The string to replace the match with.
- **string:** The original string.
- **count:** (Optional) Limits the number of replacements.
- **flags:** (Optional) Regex flags like `re.IGNORECASE`.

Example:

```
import re

text = "apple banana apple orange apple"

# Replace "apple" with "fruit"

result = re.subn(r'apple', 'fruit', text)
```

```
print("Updated text:", result[0]) # New string

print("Replacements made:", result[1]) # Number of replacements
```

Output:

Updated text: fruit banana fruit orange fruit

Replacements made: 3

Sets in Python Regular Expression

A Set contains a set of characters enclosed in square brackets []. We use them to match a single character mentioned between [].

Set	Description
[0123]	It matches the mentioned digits, i.e., 0, 1, 2, or 3.
[0-5][0-9]	It finds the match for any two-digit number between 00 and 59.
[0-9]	It will match a digit between 0 and 9.
\{n,\}	It quantifies the preceding character or group and matches a minimum of n occurrences.
\d	Matches any digit, equivalent to [0-9].
\D	Matches any non-digit character.
\w	It is used to match an alphanumeric character (a-z, A-Z, 0-9, or _).
[c-m]	It matches a lowercase alphabet between c and m.
[^cde]	It finds a match for any character except c, d, and e.
[cdef]	It will match patterns where one of the given characters (c, d, e, or f) is present.
[%&\$#@*]	Matches any special character listed (when used in [], they're not special).
[^a-zA-Z]	Matches any character not a letter (non-alphabetic).
[c-mC-M]	Matches characters from c to m (lowercase) or C to M (uppercase).

Match Object in Python RegEx

A Match Object is the result returned by regex functions like `re.search()` or `re.match()` when a pattern is found in the string. It contains details about the match, such as the matching string, its position, and the original pattern used. If no match is found, the function returns `None`.

Example:

```
import re

pattern = r"\d+"
string = "Price is 250"
match = re.search(pattern, string)

if match:
    print("Matched:", match.group())
```

Output:

```
Matched: 250
```

Here are some common attributes and methods of the match object.

Getting the Pattern and Input String from a Match Object

When a match is found using `re.search()` or `re.match()`, the match object gives us access to:

- **match.re:** Returns the regular expression (pattern object) used for the match
- **match.string:** Returns the original string that was searched

Example:

```
import re

text = "Welcome to WsCube Tech"
pattern = r"WsCube"

match = re.search(pattern, text)

if match:
    print("Matched substring:", match.group())
    print("Pattern used:", match.re.pattern)
    print("Original string:", match.string)
else:
    print("No match found.")
```

Output:

```
Matched substring: WsCube
Pattern used: WsCube
Original string: Welcome to WsCube Tech
```

Getting Index of Matched Object

When a match is found, we can get the starting and ending positions of the matched substring using:

- **match.start():** Returns the starting index of the match
- **match.end():** Returns the ending index (exclusive) of the match
- **match.span():** Returns a tuple (start, end) of the match location

Example:

```
import re

text = "Welcome to WsCube Tech"
pattern = r"WsCube"

match = re.search(pattern, text)

if match:
    print("Start index:", match.start())
    print("End index:", match.end())
    print("Span:", match.span())
else:
    print("No match found.")
```

Output:

```
Start index: 11
End index: 17
Span: (11, 17)
```

Getting Matched Substring

To get the exact matched part of the string from a regex search, use:

- **match.group():** Returns the matched substring.

Example:

```
import re

text = "Learn Python at WsCube Tech"
pattern = r"Python"

match = re.search(pattern, text)
if match:
    print("Matched substring:", match.group())
```

Output:

```
Python
```